CODEX PROJECT BRIEF

Cyber Security Project 2

Basic Encryption & Decryption

Implementation specification for a beginner-friendly Python application

Based on: DecodeLabs Industrial Training Kit – Project 2

Primary objective: Build a working application that encrypts user-entered text using a basic technique such as the Caesar cipher, decrypts the ciphertext, and displays both outputs.

This document is intended to be given to Codex as the project requirements. Follow the requirements in order and create a complete, runnable project.

1. Project overview

The project is a basic encryption and decryption application. It must demonstrate data confidentiality concepts through a reversible text transformation. Use the Caesar cipher as the default algorithm. The application is educational only and must clearly state that Caesar cipher is not secure for protecting real or sensitive information.

2. Required functionality

- Accept a text message from the user.
- Allow the user to enter a shift key (integer).
- Encrypt the message using the Caesar cipher.
- Decrypt ciphertext using the same shift key and reverse shift logic.
- Display the original text, encrypted text, and decrypted text clearly.
- Preserve spaces, punctuation, digits, and other non-alphabetic characters unchanged.
- Support uppercase and lowercase letters, including wraparound (for example, Z shifted by 3 becomes C).
- Validate the shift key and handle empty input gracefully.

3. Recommended technology and interface

Language: Python 3
Interface: Streamlit web UI
Algorithm: Caesar cipher
Editor: VS Code or any Python IDE

Create a clean, responsive, beginner-friendly interface. Include a text area for the message, a numeric shift-key input, an Encrypt action, a Decrypt action, and clearly labelled output areas. Include a Clear/Reset action and a copy-to-clipboard option if supported reliably by the chosen Streamlit implementation.

4. Caesar cipher rules

For each alphabetic character, shift its position by k places modulo 26. Preserve the character's case. For encryption use a positive shift; for decryption use the negative shift. Normalize any integer key using modulo 26.

Encryption: E(x) = (x + k) mod 26
Decryption: D(x) = (x - k) mod 26

Example: shift key = 3

Stage	Text
Original	HELLO WORLD
Encrypted	KHOOR ZRUOG
Decrypted	HELLO WORLD

5. Suggested project structure

Create a tidy project directory with at least:

```
encryption_decryption_app/
■■■ app.py
■■■ cipher.py
■■■ requirements.txt
```

■■■ `README.md`

Keep cipher logic in cipher.py and the Streamlit interface in app.py. Put required packages in requirements.txt. The README must explain prerequisites, installation, how to run the app, features, and the Caesar cipher's educational limitation.

6. Quality and usability requirements

- Use clear function names and readable, commented code suitable for a beginner.
- Separate user-interface code from encryption/decryption logic.
- Do not store or transmit user-entered messages.
- Do not claim Caesar cipher provides modern or production-grade security.
- Show useful validation messages instead of crashing on invalid input.
- Ensure the app can be launched locally using the instructions in README.md.

7. Testing requirements

- HELLO WORLD with key 3 → KHOOR ZRUOG → HELLO WORLD.
- xyz with key 3 → abc → xyz.
- Mixed case such as HeLLo with key 2 preserves case.
- Text containing spaces, punctuation, and numbers leaves those characters unchanged.
- Key 0 returns the same text.
- A key larger than 26 behaves correctly through modulo 26.
- A negative key is handled consistently.
- Empty text and invalid shift-key input are handled without an application crash.

8. Expected deliverables

- Complete runnable source code.
- requirements.txt with dependencies.
- README.md with setup and execution steps.
- Working Streamlit application.
- A small test file or documented test cases.
- Screenshots showing successful encryption and decryption.

9. Instructions for Codex

First inspect the existing workspace. If it is empty, create the project structure above. Implement the application completely, then run available tests and fix errors. Do not stop after providing sample code or a plan. At the end, report the files created, exact installation and run commands, test results, and any manual steps needed. Keep the solution aligned with the beginner-level Project 2 requirements; do not add unrelated advanced cryptographic features.

10. Acceptance checklist

- The app starts locally without errors.
- A user can enter a message and shift key.
- Encryption output is correct.
- Decryption restores the exact original message.
- Spaces, punctuation, digits, and letter case are preserved.
- Input validation works.
- README contains accurate setup and run instructions.

Source alignment note: The DecodeLabs brief explicitly requires basic encryption (Caesar cipher or similar), decryption, and display of both encrypted and decrypted output. The Streamlit interface, project structure, validation, testing, README, and screenshots are recommended implementation details added to make the submission complete and easy to demonstrate.